Sprint Retrospective

Visão geral

Reflita sobre as últimas tarefas e identifique oportunidades de melhoria seguindo as instruções para a Tática de retrospectiva.

Data	2 de novembro de 2026
Equipa	Grupo 2
Participantes	1251653 – Dinis Wang (Team Leader) 1251667 – Tomás Castro 1251665 – Nuno Magalhães 1251941 – Guilherme Pereira

Retrospetiva

Na tabela abaixo, adicione os itens: Começar a fazer, Parar de fazer e Continuar a fazer. Irão ser utilizados para falar acerca de como se pode melhorar o processo no futuro.

Começar a fazer	Parar de fazer	Continuar a fazer
- Usar uma checklist de revisão em cada pull request: quemAge na BLL, erros genéricos na interface, imports limpos, testes e título no padrão feat:/fix:	- Fazer merge antes de as alterações pedidas pelo revisor estarem feitas	- Rever todos os pull requests por outro elemento: apanharam erros reais, como um botão apagado num merge
- Proteger a main no GitHub, para que o merge só seja possível depois de uma aprovação (ação pendente da Sprint 2)	- Deixar o Javadoc, os diagramas e a consistência de nomes para o último dia	- Escrever testes unitários com uma DAL falsa, para testar a lógica sem base de dados
- Fazer logo no início da sprint as user stories de consistência (nomes das classes e diagramas), antes das restantes e a cargo do Team Leader	- Concentrar o trabalho nos últimos dias: houve user stories a fechar no dia da Review	- Manter os critérios de aceitação alinhados com o Trello e a checklist de codificação (ecrã, testes e Javadoc)
- Escrever o Javadoc e atualizar os diagramas em cada user story, e não só no fim da sprint	- Abrir uma janela nova para cada ecrã: passar a uma só janela, com menu e painéis	- Seguir o Git flow: uma branch por user story, nomes feat-/fix-, merge commit e commits claros
- Preparar a demonstração antes da Review: rede, base de dados, ficheiros XML e dados de teste	- Abrir pull requests sem testes da lógica de negócio (aconteceu na gestão de quartos)	- Ajudar os colegas a concluir as suas user stories quando terminamos as nossas
- Juntar as operações inserir, listar, atualizar e apagar num só teste completo por entidade, com limpeza dos dados mesmo quando o teste falha	- Reutilizar branches já fundidas ou misturar a Quality numa branch de funcionalidade	- Guardar evidências do planeamento (capturas do quadro) na Sprint Planning
- Confirmar com o docente, antes da sprint, os atores das user stories que ficam por esclarecer		- Criar a reserva a partir dos objetos cliente e quarto, e não de identificadores soltos

Resumo

A Sprint 3 correu bem: a equipa concluiu as user stories de reservas (criar, cancelar, alterar datas e consultar as de um cliente), a gestão de quartos e de serviços extra, a importação das entregas dos fornecedores em XML e as duas user stories extra de sugestões e queixas. Foram 22 dos 26 pontos comprometidos, mais os 4 pontos extra, todas revistas em pull request, com 276 testes unitários a passar. Os diagramas de classes foram atualizados ao código e organizados por camada (modelo, DAL, BLL e controllers), e o Javadoc foi acrescentado às classes das user stories da sprint. Reconhecemos que, tal como na Sprint 2, parte do trabalho ficou concentrada nos últimos dias, que a consistência de nomes entre camadas (US32) e o diagrama de sequência detalhado (US33) não ficaram integrados durante a sprint, e que alguns pull requests foram fundidos antes de as correções pedidas pelo revisor estarem feitas. Na Sprint Review, a demonstração mostrou as funcionalidades a funcionar e o docente deixou sugestões que vamos aplicar na Sprint 4: reorganizar a interface numa única janela com menu e painéis, simplificar a cobrança dos serviços e ligá-los à reserva, validar as entregas em duas etapas e juntar os testes à base de dados num teste completo por entidade. Com estas ações, queremos planear melhor o trabalho e distribuí-lo de forma mais equilibrada ao longo da semana.

Feedback da Sprint Review

• Na demonstração funcionaram a gestão de serviços extra (criar, editar e desativar), a criação de reservas (com pesquisa do cliente pelo NIF e validação das datas), o registo de sugestões e queixas e a importação das entregas em XML, com validação do ficheiro e aviso quando as contas não coincidem (por exemplo, um total de 160,54 € calculado contra 160,59 € no ficheiro).
• As duas user stories extra, registar e consultar sugestões e queixas, ficaram concluídas.
• As user stories de consistência (nomes das classes e diagramas) devem ser feitas logo no início da sprint, antes das restantes, para não gerar confusão. Podem ficar a cargo do Team Leader.
• A aplicação abre várias janelas ao mesmo tempo. O docente sugeriu uma única janela com um menu e painéis que se atualizam (por exemplo, clientes e serviços), com um botão para criar que abre um formulário e atualiza a lista, e a edição feita ao selecionar o registo na tabela. As reservas já têm submenu, e a reorganização fica como objetivo da próxima sprint. É preciso investigar como o JavaFX gere estas janelas.
• Cobrança dos serviços: o docente sugeriu simplificar, aplicando aos serviços o mesmo conceito do preço por noite do quarto (por exemplo, o ginásio a um preço por dia), para facilitar o cálculo do total da reserva. A equipa concordou que é preferível simplificar.
• Os serviços ainda não entram na reserva. O cliente pode pedi-los no momento da reserva ou mais tarde, durante a estadia, e o valor deve somar-se ao total. Também é preciso rever se o early check-in e o late check-out são serviços ou taxas.
• Entregas em XML: o docente sugeriu que o funcionário carregue o ficheiro e que só depois o gestor ou o rececionista o valide (aceitando ou recusando), dando a entrada em stock apenas nessa altura. Hoje a entrada fica registada em nome de quem a aceita.
• A forma como a reserva é criada, a partir dos objetos cliente e quarto e dos seus identificadores, em vez de ids soltos, foi considerada adequada.
• Testes à base de dados: cada entidade deve ter um teste completo de inserir, listar, atualizar e apagar, que pode juntar estas operações num só teste para os ficheiros não ficarem tão extensos, mantendo à parte os testes de outras regras (por exemplo, o cálculo do preço). A limpeza dos dados tem de ser garantida mesmo quando um teste falha a meio, para não ficarem registos na base de dados.
• Na demonstração, a primeira tentativa de importar o XML falhou por falta de ligação à rede e de acesso ao ficheiro, e foi difícil recordar o NIF do cliente de teste. A demonstração deve ser preparada antes da Review.

✅ Ações a tomar

- Aplicar a checklist de revisão a todos os pull requests (quemAge, erros genéricos, imports, testes e título) e só fazer merge depois da aprovação do revisor
- Ativar a proteção da main no GitHub (a pedir ao Guilherme, como dono do repositório)
- Fazer logo no início da sprint as user stories de consistência (nomes das classes e diagramas) e escrever o Javadoc em cada user story, em vez de o deixar para o fim
- Reorganizar a interface numa única janela, com menu e painéis que se atualizam, e investigar como o JavaFX gere as janelas
- Simplificar a cobrança dos serviços (preço por noite ou por dia) e permitir juntá-los à reserva, na criação ou mais tarde, com o valor somado ao total
- Rever o fluxo das entregas: o funcionário carrega o ficheiro e o gestor ou o rececionista valida-o antes da entrada em stock.
- Juntar os testes à base de dados num teste completo por entidade (inserir, listar, atualizar e apagar) e garantir a limpeza dos dados mesmo quando o teste falha

- Criar a área de check-in e check-out, com validação da disponibilidade do quarto para o early check-in e o late check-out
- Acrescentar o estado das entregas (aprovada ou recusada) e definir o que acontece a uma entrega recusada
- Concluir o que ficou pendente: consistência de nomes BLL/DAL (US32), diagrama de sequência detalhado (US33), testes e quemAge na gestão de quartos (US31) e a segunda parte do Javadoc
- Confirmar com o docente quem consulta as sugestões e queixas, quem pode alterar as datas de uma reserva e se o early check-in e o late check-out são serviços ou taxas
- Preparar a demonstração antes da Review (rede, base de dados, ficheiros XML e dados de teste) e definir uma meta intermédia na Daily Scrum